

INTERNET OF THINGS (IoT) LABORATORY

COMPLETE HANDWRITTEN PRACTICAL RECORD CONTENT GUIDE

Department of Computer Science and Engineering / Information Technology

Academic Session: 2025–2026 • Degree: Bachelor of Technology (B.Tech)

Notice for CSE Students (What to Write by Hand): This document contains **strictly the text, theory, mathematical derivations, algorithms, code logic explanations, results, and viva-voce answers** that you need to handwrite on the right-hand ruled pages of your practical lab copy. All diagrams, circuit schematics, hardware photos, full C++ codes, timing waveforms, and observation tables are attached on the facing left-hand unruled pages from your *Printable Attachments Booklet*.

Experiment 01: Overview of IoT Architecture, Characteristics & Constrained Protocol Engineering

Experiment No.: 01	Course: Internet of Things Laboratory	Semester: VI / VII
--------------------	---------------------------------------	--------------------

1. Aim & Objectives

- To study the foundational paradigm, definitions, evolutionary history, and architectural frameworks of the Internet of Things (IoT).
- To rigorously analyze the ten core characteristics defining modern cyber-physical IoT systems.
- To understand the 5-tier hierarchical architectural reference model from physical sensing to cloud business analytics.
- To perform an engineering comparison of constrained application-layer protocols (MQTT, CoAP, HTTP/REST, WebSocket, AMQP) with respect to packet overhead, transport mechanisms, connection persistence, and Quality of Service (QoS).
- To evaluate edge computing versus centralized cloud processing trade-offs in low-power constrained environments.

2. Theoretical Background & Foundations

A. Genesis and Standardized Definitions of IoT

The term *Internet of Things* was formally coined in 1999 by Kevin Ashton during work on RFID supply-chain infrastructure. IoT represents the convergence of Ubiquitous Computing (Mark Weiser), Embedded Systems, Wireless Sensor Networks (WSN), and Cloud Computing.

- **ITU-T Recommendation Y.2060 Definition:** “A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual ‘things’ have identities, physical attributes, and virtual personalities, use intelligent interfaces, and are seamlessly integrated into the information network.”
- **IEEE IoT Working Group Definition:** “A self-configuring, adaptive, complex network that interconnects ‘things’ to the Internet through the use of standard communication protocols, where things can be physical objects or virtual representations.”
- **Cyber-Physical System (CPS) vs. IoT:** While CPS emphasizes the closed-loop computational feedback coupling between digital algorithms and physical processes (such as robotics and smart power grids), IoT provides the global networking fabric, data serialization, and cloud telemetry infrastructure.

B. Detailed Analysis of the Ten Core Characteristics of IoT

1. **Interconnectivity & Global Addressing:** Any physical object can be assigned a unique logical address (IPv6 / 6LoWPAN) and integrated into the global communication infrastructure.
2. **Things-Related Services & Virtual Digital Twins:** The physical state of a sensor (temperature, pressure, velocity) is semantically mirrored into a digital data structure (JSON/CBOR object) in the cloud or gateway.

3. **Heterogeneity across Hardware & Software:** Interoperability across 8-bit MCUs (ATmega328P), 32-bit SoCs (ESP32, ARM Cortex-M), diverse RTOS environments (FreeRTOS, Zephyr), and multi-vendor fieldbuses.
4. **Dynamic Changes in State & Context Awareness:** IoT nodes dynamically alternate between deep-sleep modes ($< 15 \mu\text{A}$) and active radio bursts, dynamically adapting to fluctuating network topologies and battery levels.
5. **Massive Scale & Concurrency:** Networks must handle orders of magnitude more connected endpoints than the human Internet, requiring non-blocking asynchronous brokers (e.g., MQTT Mosquitto, EMQX).
6. **Embedded Edge Intelligence & Fog Computing:** Offloading data preprocessing, threshold comparisons, Kalman filtering, and inference to local microcontrollers reduces bandwidth consumption, network egress costs, and cloud latency.
7. **Self-Configuration & Autonomous Self-Healing:** Mesh networks (Zigbee, RPL) dynamically re-route data packets around failing nodes, while Firmware Over-The-Air (FOTA) delivers cryptographic binary patches.
8. **Open Standards & Protocol Interoperability:** Built entirely on open IETF, IEEE, OASIS, and ISO standards (IEEE 802.15.4, RFC 7252 CoAP, OASIS MQTT, RFC 6749 OAuth 2.0).
9. **End-to-End Cryptographic Security:** Implementing the CIA Triad (Confidentiality, Integrity, Availability) on constrained hardware using hardware-accelerated AES-128/256, Elliptic Curve Cryptography (ECC), and TLS/DTLS.
10. **Data-Driven Value Creation & Stream Ingestion:** Raw time-series telemetry streams are transformed into actionable predictive maintenance, anomaly detection, and operational automation via Kafka, InfluxDB, and ML pipelines.

3. 5-Tier Hierarchical IoT Architecture Reference Framework

1. **Layer 1: Physical Environment:** The real-world context containing continuous physical phenomena (temperature gradients, relative humidity, pressure, acoustic vibration).
2. **Layer 2: Perception / Sensing Layer:** Transducers convert physical parameters into analog voltages. The microcontroller's Successive Approximation ADC quantizes signals into digital values and performs signal conditioning.
3. **Layer 3: Network / Transport Layer:** Responsible for secure packet routing across short-range WPAN (802.15.4, BLE, Zigbee), LPWAN (LoRaWAN, NB-IoT), or WLAN (Wi-Fi 802.11) using IPv6/6LoWPAN.
4. **Layer 4: Processing / Middleware Layer:** Ingests streaming data through message brokers (MQTT Mosquitto), validates modulo parity checksums, normalizes schema, and stores time-series metrics in databases (InfluxDB, MongoDB).
5. **Layer 5: Business / Application Layer:** Provides real-time interactive user interfaces, mobile applications, SCADA supervisory control, predictive failure models, and enterprise ERP integrations.

4. Constrained Protocol Engineering Analysis

- **MQTT (Message Queuing Telemetry Transport):** An OASIS standard binary protocol operating over TCP (port 1883/8883) utilizing an asynchronous Publish/Subscribe model. It features an ultra-low fixed 2-byte header, persistent socket connections, Last Will and Testament (LWT) for fault detection, and 3 configurable Quality of Service (QoS) delivery tiers. Ideal for high-latency, low-bandwidth telemetry.
- **CoAP (Constrained Application Protocol):** Defined in RFC 7252, CoAP operates over UDP (port 5683/5684) and brings the RESTful request/response paradigm (GET, POST, PUT, DELETE) to ultra-constrained 8-bit MCUs. It uses a compact 4-byte binary header, supports UDP multicast, and provides asynchronous resource observation with optional Confirmable (CON) / Non-Confirmable (NON) reliability.
- **HTTP/REST (Hypertext Transfer Protocol):** The conventional Web protocol over TCP (port 80/443). While universally interoperable with enterprise cloud services, its large text headers (> 500 bytes) and mandatory TCP 3-way handshake overhead make it unsuited for battery-powered edge sensor nodes.
- **WebSocket:** Operates over TCP (port 80/443) providing full-duplex, persistent bidirectional communication channels after an initial HTTP handshake. Frame overhead is minimal (2 – 10 bytes), making

it ideal for live web browser telemetry dashboards without polling overhead.

A. MQTT Quality of Service (QoS) Deep Dive

- **QoS 0 (At most once):** Fire-and-forget; message dispatched without acknowledgment. Zero network retry overhead.
- **QoS 1 (At least once):** Publisher sends packet and awaits 'PUBACK'. Retransmits if unacknowledged (duplicate delivery possible).
- **QoS 2 (Exactly once):** Four-step cryptographic handshake ('PUBLISH' → 'PUBREC' → 'PUBREL' → 'PUBCOMP') guarantees exactly-once delivery without loss or duplication.

5. Architectural Review & Design Trade-Offs

- **Bandwidth vs. Energy vs. Range:** Shannon-Hartley theorem dictates that operating over long distances with minimal transmit power (e.g., LoRaWAN) requires reducing channel bandwidth and bit rates.
- **Edge Computing vs. Cloud Dependency:** Performing threshold comparisons locally eliminates network latency ($\approx 1 - 5$ ms vs. $100 - 500$ ms for cloud round-trip) and preserves system availability during network outages.

6. Viva-Voce Questions & Detailed Answers

1. **Q: Explain the publish-subscribe pattern used in MQTT and why it is superior to HTTP polling for IoT.**

Ans: Publish-subscribe decouples spatial and temporal dependencies between data producers (publishers) and consumers (subscribers) through a central broker. HTTP polling wastes cellular/battery bandwidth by repeatedly establishing 3-way TCP handshakes and sending heavy text headers even when no new data exists. MQTT keeps a persistent lightweight socket open and pushes data only when state changes occur.

2. **Q: What is 6LoWPAN and how does it solve IPv6 packet encapsulation on constrained radios?**

Ans: 6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) defines encapsulation and header compression mechanisms (RFC 6282) that compress standard 40-byte IPv6 headers down to 2–7 bytes, allowing 1280-byte IPv6 MTU packets to be fragmented and transmitted across 127-byte IEEE 802.15.4 physical radio frames.

3. **Q: What is the function of MQTT's Last Will and Testament (LWT) feature?**

Ans: When a client connects to the broker, it registers an LWT topic and payload. If the client disconnects ungracefully (e.g., battery depletion or hardware failure), the broker detects keep-alive ping timeout and automatically publishes the LWT message to alert subscribers of the node's offline status.

Experiment 02: Arduino UNO Architecture, AVR Subsystems & Dual LED Interfacing

Experiment No.: 02 Topic: Embedded Systems Architecture & GPIO Interfacing Course: IoT Lab

1. Aim & Objectives

- To investigate the internal hardware subsystems, memory map, register architecture, and power management of the ATmega328P microcontroller on the Arduino UNO board.
- To interface two discrete semiconductor LEDs (Green and Red) to GPIO Pins D8 and D7 using a solderless breadboard.
- To mathematically derive the required series current-limiting resistor values using semiconductor bandgap equations ($E_g = \frac{hc}{\lambda}$), Kirchhoff's Voltage Law (KVL), and Ohm's Law.
- To write, compile, flash, and verify an embedded C++ program that drives the LEDs in periodic anti-phase switching at 1.0 s intervals.
- To analyze digital logic transitions, push-pull driver stages, and compare synchronous blocking vs. asynchronous interrupt timing.

2. Hardware Apparatus Required & Specifications

1. **Arduino UNO R3 Board:** ATmega328P 8-bit AVR RISC MCU, 16 MHz Crystal Resonator, 5V Operating Logic, 32 KB Flash.
2. **Green LED (5 mm):** Gallium Phosphide (GaP) substrate, $\lambda \approx 525$ nm, Forward Voltage $V_F \approx 2.1$ V, $I_{\max} = 20$ mA.
3. **Red LED (5 mm):** Gallium Arsenide Phosphide (GaAsP), $\lambda \approx 625$ nm, Forward Voltage $V_F \approx 1.9$ V, $I_{\max} = 20$ mA.
4. **Current-Limiting Resistors:** 1 k Ω (or 220 Ω), 0.25 W Carbon Film Resistors, $\pm 5\%$ Tolerance (2 units).
5. **Solderless Breadboard:** Standard 830 tie-point breadboard with dual power distribution rails.
6. **Jumper Wires & Cable:** 24 AWG solid-core male-to-male wires and USB Type-A to Type-B programming cable.

3. In-Depth ATmega328P Microcontroller Architecture Breakdown

1. **Harvard Architecture Core:** Features separate instruction (Flash) and data (SRAM) memory buses, allowing single-cycle execution of instructions with pipelining at 16 MHz ($T_{\text{clk}} = 62.5$ ns).
2. **Memory Organization:**
 - **Flash Memory (32 KB):** Non-volatile memory for program instructions. Divided into Application Flash Section and Bootloader Section (0.5 KB used by Optiboot).
 - **SRAM (2 KB):** Volatile memory partitioned into Data Segment, BSS (uninitialized variables), Dynamic Heap, and Call Stack (which grows downward).
 - **EEPROM (1 KB):** Non-volatile byte-addressable storage with 100,000 write cycle endurance for calibration constants.
3. **GPIO Port Architecture & Registers:** Each digital pin is controlled by three 8-bit memory-mapped I/O registers:
 - 'DDRx' (Data Direction Register): Sets pin as Input ('0') or Output ('1').
 - 'PORTx' (Data Register): Sets output state HIGH ('1') or LOW ('0'), or activates internal 20 – 50 k Ω pull-up when configured as input.
 - 'PINx' (Input Pins Address): Reads the physical digital logic level on the pin.
4. **Push-Pull Output Stage Physics:** When configured as an output, the pin is driven by complementary MOSFETs: a high-side P-MOSFET connects the pin to +5 V (source mode), and a low-side N-MOSFET connects it to GND (sink mode). Internal on-resistance is $\approx 25 \Omega$.
5. **Analog-to-Digital Converter (ADC):** 10-bit Successive Approximation Register (SAR) ADC with 6 multiplexed channels (A0–A5). Quantization step: $Q = \frac{V_{\text{ref}}}{2^{10}} = \frac{5.0 \text{ V}}{1024} = 4.88 \text{ mV/count}$.
6. **Clock & Power Subsystems:** 16 MHz crystal resonator, onboard NCP1117ST50 linear 5V regulator, LP2985 3.3V regulator, and 500 mA self-resetting USB polyfuse.

4. Theoretical Principles & Mathematical Derivations

A. LED Semiconductor Bandgap Physics

An LED is a heavily doped P - N junction diode fabricated from direct bandgap compound semiconductors. When forward-biased, injected electron-hole pairs recombine across the bandgap E_g , emitting photons with wavelength:

$$E_g = h \cdot \nu = \frac{h \cdot c}{\lambda} \implies \lambda = \frac{h \cdot c}{E_g} \quad (1)$$

Where $h = 6.626 \times 10^{-34} \text{ J} \cdot \text{s}$ (Planck's constant) and $c = 3.0 \times 10^8 \text{ m/s}$ (Speed of light).

B. Resistor Derivation via Kirchhoff's Voltage Law (KVL) & Ohm's Law

Applying KVL around the closed loop containing the GPIO output stage, current-limiting resistor R , and LED:

$$V_{CC} - V_R - V_F = 0 \implies V_{CC} - (I_F \cdot R) - V_F = 0 \implies R = \frac{V_{CC} - V_F}{I_F} \quad (2)$$

For $V_{CC} = 5.0 \text{ V}$, $V_F \approx 2.0 \text{ V}$, and safe target current $I_F = 15 \text{ mA} = 0.015 \text{ A}$:

$$R_{\text{calc}} = \frac{5.0 \text{ V} - 2.0 \text{ V}}{0.015 \text{ A}} = \frac{3.0 \text{ V}}{0.015 \text{ A}} = 200 \Omega \quad (3)$$

Standard commercial selections:

- **Using $R = 220 \Omega$:** $I_F = \frac{5.0-2.0}{220} = 13.63 \text{ mA}$ (Maximum safe brightness).
- **Using $R = 1 \text{ k}\Omega$:** $I_F = \frac{5.0-2.0}{1000} = 3.0 \text{ mA}$ (Conservative low-power drive).
- **Resistor Power Dissipation:** $P_R = I_F^2 \cdot R = (0.003)^2 \times 1000 = 9 \text{ mW} \ll 0.25 \text{ W}$ (Safe margin).

5. Step-by-Step Procedure & Algorithm

1. Insert the Green and Red LEDs into isolated columns on the solderless breadboard.
2. Connect a $1 \text{ k}\Omega$ current-limiting resistor in series between the anode of each LED and Arduino digital pins D8 (Green) and D7 (Red).
3. Tie both LED cathodes (short legs/flat notch) to the common ground rail of the breadboard, and connect the rail to the Arduino GND pin.
4. Connect the Arduino UNO to the host workstation via USB programming cable.
5. Open Arduino IDE, select **Arduino Uno** under Board menu, and configure the appropriate COM port.
6. Compile and upload the C++ firmware binary via the AVR Optiboot bootloader.
7. Observe the periodic anti-phase toggling of the LEDs at exact 1000 ms intervals.

6. Firmware Execution Logic & Software Flow Analysis

- **Hardware Pin Configuration ('setup()')**: Pins D8 and D7 are configured as output drivers using 'pinMode(pin, OUTPUT)'. This modifies the ATmega328P Data Direction Registers ('DDRB' bit 0 for D8, and 'DDRD' bit 7 for D7) by writing bitmask '1'. Both pins are initialized to logic LOW (0 V).
- **Phase 1 (Active High Drive)**: In the infinite 'loop()', Pin D8 is set to HIGH (+5 V) via 'digitalWrite(8, HIGH)' while Pin D7 is set to LOW (0 V). Current flows from Pin D8 through R_1 into Green LED, turning it ON ($I_{F1} \approx 3.0 \text{ mA}$). Red LED remains reverse/zero biased (OFF). The CPU then halts execution for 1000 ms via 'delay(1000)'.
- **Phase 2 (Anti-Phase State Inversion)**: Pin D8 is driven LOW (0 V) extinguishing Green LED, while Pin D7 is driven HIGH (+5 V) turning Red LED ON ($I_{F2} \approx 3.0 \text{ mA}$). The system executes another 1000 ms delay before restarting Phase 1.

7. Result & Discussion

The Arduino UNO GPIO pins D8 and D7 successfully drove the external Green and Red semiconductor LEDs in continuous periodic anti-phase switching at exact 1.0 s (1000 ms) intervals. Ohm's Law and KVL calculations were experimentally verified through multimeter current measurement, maintaining safe forward current levels ($I_F \approx 3.0 \text{ mA} < 20 \text{ mA}$).

8. Viva-Voce Questions & Detailed Answers

1. **Q: What is the difference between Direct Port Manipulation and the Arduino 'digitalWrite()' API?**

Ans: 'digitalWrite()' introduces $\approx 50 - 60$ machine clock cycles ($\approx 3.5 \mu\text{s}$) of execution overhead due to

array lookups, pin mapping, timer disable checks, and interrupt safety masks. Direct port manipulation (e.g., `PORTB |= (1 << PB0);`) compiles to a single 1-cycle assembly instruction (`'sbi'`, 62.5 ns), achieving maximum switching speed.

2. **Q: What is the maximum current rating for an ATmega328P pin and the entire chip?**

Ans: Absolute maximum DC current per individual I/O pin is 40 mA (20 mA recommended). The cumulative sum of all sourced or sunk current through V_{CC} and GND pins must not exceed 200 mA to prevent package thermal destruction.

3. **Q: Why is `'delay()'` considered bad practice in real-time embedded systems, and what is the alternative?**

Ans: `'delay()'` is a blocking busy-wait loop that traps the CPU, preventing it from executing sensor sampling, handling serial communication, or responding to external events. The alternative is non-blocking timer polling using `'millis()'` with a finite state machine (FSM) or hardware timer interrupts.

Experiment 03: DHT11 Digital Sensor Interfacing, 1-Wire Protocol Decoding & Closed-Loop Temperature Alert System

Experiment No.: 03 Topic: Single-Bus Telemetry Decoding & Closed-Loop Control Course: IoT Lab

1. Aim & Objectives

- To interface the DHT11 digital relative humidity and temperature sensor with the Arduino UNO (Pin D7).
- To decode the proprietary single-bus 1-Wire asynchronous communication protocol at the microsecond level.
- To verify data integrity using 8-bit modulo-256 parity checksum arithmetic.
- To implement a closed-loop automated feedback control system that actuates a high-temperature Alert LED (Pin D8) when $T \geq 30.0^\circ\text{C}$.
- To stream structured telemetry logs to the Arduino Serial Monitor at 9600 Baud and evaluate sensor failure modes.

2. Hardware Apparatus Required & Specifications

1. **Arduino UNO R3 Board:** ATmega328P MCU, 16 MHz Clock, 5V Operating Logic.
2. **DHT11 Digital Module:** Temperature Range: $0 - 50^\circ\text{C}$ ($\pm 2^\circ\text{C}$ accuracy), Relative Humidity Range: $20 - 90\%$ ($\pm 5\%$), Single-Bus Digital Output.
3. **Red Alert Indicator LED:** 5 mm GaAsP LED, $V_F \approx 1.9\text{ V}$, $I_{\max} = 20\text{ mA}$.
4. **Current-Limiting Resistor:** $220\ \Omega$ (or $1\text{ k}\Omega$), 0.25 W Carbon Film Resistor.
5. **Solderless Breadboard & Jumpers:** 830 tie-point breadboard and solid-core connecting wires.
6. **Controlled Thermal Stimulus:** Local thermal excitation source (fingertip friction / proximity lamp).

3. Physical Transduction Mechanisms & Sensor Physics

1. **Relative Humidity Sensing Substrate:** Utilizes a hygroscopic polymer capacitive/resistive substrate. Water vapor molecules from ambient air are adsorbed onto the dielectric polymer surface, altering its ionic conductivity in direct proportion to ambient relative humidity:

$$\text{RH}(\%) = \frac{P_{\text{actual}}}{P_{\text{saturated}}} \times 100\% \quad (4)$$

2. **Negative Temperature Coefficient (NTC) Thermistor:** The temperature-sensing element is a semiconductor ceramic thermistor. Thermal excitation frees valence electrons across the semiconductor bandgap, exponentially decreasing the electrical resistance according to the Steinhart-Hart equation:

$$R(T) = R_0 \cdot \exp \left[\beta \left(\frac{1}{T} - \frac{1}{T_0} \right) \right] \quad (5)$$

Where $R_0 = 10\text{ k}\Omega$ at $T_0 = 298.15\text{ K}$ (25°C) and $\beta \approx 3950\text{ K}$ is the material constant.

4. Single-Bus (1-Wire) Communication Handshake Protocol

The DHT11 uses an open-drain bidirectional signal line pulled HIGH to $+5\text{ V}$ via a $4.7 - 10\text{ k}\Omega$ resistor. Transmission proceeds through distinct sequential timing phases:

1. **Bus Idle State:** The bus is held continuously at logic HIGH ($+5\text{ V}$).
2. **Host Start Signal:** Arduino asserts a logic LOW on DATA for $\geq 18\text{ ms}$ (typically 20 ms) to wake up the sensor OTP microcontroller, then releases the bus and drives HIGH for $20 - 40\ \mu\text{s}$.
3. **Sensor Acknowledgment:** DHT11 pulls the bus LOW for $80\ \mu\text{s}$, followed by driving it HIGH for $80\ \mu\text{s}$.
4. **40-Bit Data Transmission (5 Bytes MSB-First):**
 - Every bit transmission starts with a standard $50\ \mu\text{s}$ LOW pulse.
 - **Logic '0' Encoding:** The following HIGH pulse duration lasts $26 - 28\ \mu\text{s}$.
 - **Logic '1' Encoding:** The following HIGH pulse duration lasts $70\ \mu\text{s}$.

5. Frame Structure & Checksum Verification Formula:

$$\text{Frame} = [\text{Byte 1: RH}_{\text{int}}] + [\text{Byte 2: RH}_{\text{dec}}] + [\text{Byte 3: } T_{\text{int}}] + [\text{Byte 4: } T_{\text{dec}}] + [\text{Byte 5: Checksum}] \quad (6)$$

$$\text{Checksum Byte} = (\text{Byte 1} + \text{Byte 2} + \text{Byte 3} + \text{Byte 4}) \pmod{256} \quad (7)$$

If the calculated sum does not equal Byte 5, the packet is corrupted and discarded.

5. Closed-Loop Feedback Control Theory

The system implements a discrete bang-bang feedback control loop:

$$u(t) = \begin{cases} 1 \text{ (ALERT ON)}, & \text{if } T(t) \geq 30.0^\circ\text{C} \\ 0 \text{ (NORMAL OFF)}, & \text{if } T(t) < 30.0^\circ\text{C} \end{cases} \quad (8)$$

In industrial implementations, a small hysteresis margin ($\Delta T = 0.5^\circ\text{C}$) is introduced to prevent relay/LED chattering around the boundary setpoint.

6. Algorithm & Firmware Execution Flow Analysis

- **Sensor Initialization ('setup()')**: The serial UART interface is initialized at 9600 Baud via 'Serial.begin(9600)'. Digital Pin 8 is configured as output for the Alert LED and driven LOW. The DHT sensor class is initialized via 'dht.begin()', configuring Pin D7 for 1-Wire communication.
- **Sampling Rate Constraint ('loop()')**: The DHT11 internal sensing elements require at least 1.0 – 2.0s between successive conversions. The firmware enforces a 2000ms delay ('delay(2000)') between sampling iterations.
- **Frame Acquisition & 'isnan()' Validation**: The microcontroller reads humidity and temperature using 'dht.readHumidity()' and 'dht.readTemperature()'. The firmware checks 'isnan(val)' to detect communication timeouts or parity errors before processing.
- **Closed-Loop Threshold Evaluation**: The temperature reading is compared against setpoint $T_{\text{threshold}} = 30.0^\circ\text{C}$. If $T \geq 30.0^\circ\text{C}$, Pin D8 is set HIGH turning the Red Alert LED ON; otherwise, it is driven LOW (Normal state).
- **Serial Telemetry Formatting**: Current system timestamp ('millis() / 1000'), humidity, temperature, and alert state are serialized into tab-delimited ASCII strings and transmitted over UART to the host Serial Monitor.

7. Result & Discussion

The DHT11 digital transducer successfully transmitted real-time relative humidity and ambient temperature measurements to the Arduino UNO over the single-bus line. 8-bit modulo-256 parity checksum calculations verified zero packet corruption. The closed-loop control system automatically triggered the alert LED at 30.0°C and deactivated it once temperature dropped below the setpoint, demonstrating stable bang-bang control.

8. Viva-Voce Questions & Detailed Answers

1. **Q: How does the DHT11 represent logic '0' versus logic '1'?**
Ans: Both bit transmissions begin with a $50\mu\text{s}$ LOW pulse. The subsequent HIGH duration distinguishes the bit: a $26 - 28\mu\text{s}$ HIGH duration represents logic '0', whereas a $70\mu\text{s}$ HIGH duration represents logic '1'.
2. **Q: Why is a pull-up resistor required on the DHT11 DATA line?**
Ans: The DHT11 uses an open-drain output stage. The pull-up resistor ($4.7 - 10\text{k}\Omega$) holds the bus at logic HIGH (5V) during idle periods and prevents the signal line from floating.
3. **Q: What causes 'NaN' errors in Arduino DHT sensor readings?**
Ans: 'NaN' (Not a Number) occurs when the sensor fails to respond to the host start signal within the expected $80\mu\text{s}$ window, when communication times out, or when the 8-bit checksum calculation fails due to loose breadboard wiring.